

Lab Instructions

In this lab, we focus on **data-parallel training from first principles** using NumPy logistic regression and PyTorch DDP.

This lab builds directly on the lab of yesterday: **broadcast** initializes model weights, **scatter** partitions data, and **all-reduce** averages gradients so every model replica applies the same update. This matches the core data-parallel idea described in PyTorch's distributed overview: model replicas process different data samples and average gradients before optimizer steps. PyTorch DDP automates this by using `torch.distributed` collectives to synchronize gradients and buffers across processes. [\[github.com\]](#) [\[multiproce...thedocs.io\]](#)

Full Example Code: July29_data_parallel_training.py

What Students Should Observe

Students should usually observe:

1. **Gradient averaging** produces results close to serial training because averaged local gradients approximate the gradient over the full dataset.
2. **World size** changes the size of each local shard.
3. More workers do not automatically make the simulation faster, especially because this version is not actually running the local gradient computations in parallel.
4. **Communication rounds** increase with every synchronized epoch.
5. **Communication scalars** increase with world size and model size.
6. **Weight averaging** may behave differently from gradient averaging, especially when `local_steps` is large.

This mirrors the main idea of real data-parallel training: each model replica computes gradients on different samples, gradients are averaged, and replicas apply synchronized updates. PyTorch DDP automates the synchronization by using collectives during backward computation. [\[github.com\]](#) [\[multiproce...thedocs.io\]](#)

Student Results Table

Students should create a table like this:

Method	Time	Final Loss	Train Acc	Test Acc	Comm Rounds	Comm Scalars
serial					0	0
grad_avg_world_1						
grad_avg_world_2						
grad_avg_world_4						
grad_avg_world_8						
weight_avg_world_4_steps_1						
weight_avg_world_4_steps_5						

Required Chart

Students should make at least one chart:

x-axis: epoch

y-axis: training loss

series: serial, grad_avg_world_2, grad_avg_world_4, grad_avg_world_8

Optional second chart:

x-axis: method

y-axis: communication_scalars

Discussion Questions

1. Why should data-parallel replicas start from the same weights?
2. Why does each worker train on different data?
3. What does all-reduce do in gradient averaging?

4. Why does every worker need the same averaged gradient?
 5. How is gradient averaging different from weight averaging?
 6. Why might weight averaging with many local steps behave differently?
 7. What is the relationship between:
 - world size
 - local batch size
 - global batch size
 8. What happens as the model gets larger?
 9. What happens as communication gets slower?
 10. How does this connect to PyTorch DDP?
-

Extension A: True Parallel Local Gradient Computation

The main lab simulates data-parallel training sequentially for clarity. Advanced students can parallelize the local gradient computation using `ProcessPoolExecutor`.

Add this worker:

```
from concurrent.futures import ProcessPoolExecutor
```

```
def local_loss_grad_worker(args):
```

```
    """
```

```
    Worker for parallel local gradient computation.
```

```
    """
```

```
    params, X_shard, y_shard = args
```

```
    loss, grads = loss_and_grad(
```

```
        params,
```

```
X_shard,  
y_shard,  
)
```

```
return loss, grads
```

Then use:

```
def train_data_parallel_with_processes(  
    X_train,  
    y_train,  
    X_test,  
    y_test,  
    world_size=4,  
    lr=LEARNING_RATE,  
    epochs=EPOCHS,  
):
```

```
    """
```

Data-parallel training where local gradient computation is
actually parallelized with ProcessPoolExecutor.

This is closer to a true multi-process simulation, but still
not real DDP.

```
    """
```

```
    params = init_params(X_train.shape[1])
```

```
    shards = shard_data(X_train, y_train, world_size)
```

```
history = []

n_params = count_parameters(params)

communication_rounds = 0
communication_scalars = 0

start = time.perf_counter()

with ProcessPoolExecutor(max_workers=world_size) as executor:
    for epoch in range(epochs):
        tasks = [
            (
                copy_params(params),
                shard["X"],
                shard["y"],
            )
            for shard in shards
        ]

        outputs = list(
            executor.map(
                local_loss_grad_worker,
                tasks,
            )
        )
```

```
local_losses = [  
    out[0]  
    for out in outputs  
]
```

```
local_grads = [  
    out[1]  
    for out in outputs  
]
```

```
avg_grads = average_gradients(local_grads)
```

```
communication_rounds += 1
```

```
communication_scalars += world_size * n_params
```

```
apply_gradients(params, avg_grads, lr)
```

```
history.append({  
    "method": f"process_grad_avg_world_{world_size}",  
    "epoch": epoch,  
    "train_loss": float(np.mean(local_losses)),  
    "train_acc": accuracy(params, X_train, y_train),  
    "test_acc": accuracy(params, X_test, y_test),  
    "communication_rounds": communication_rounds,  
    "communication_scalars": communication_scalars,  
})
```

```
elapsed = time.perf_counter() - start
```

```
return params, history, elapsed
```

Discussion:

Did true multiprocessing speed things up?

Did pickling params and shards add overhead?

How would real DDP reduce this overhead?

Extension B: Non-IID Data Shards

This extension connects to federated learning.

```
def shard_data_non_iid(X, y, world_size):
```

```
    """
```

Create non-IID shards by sorting by label first.

This gives some workers mostly one class and other workers mostly another class.

```
    """
```

```
    idx = np.argsort(y)
```

```
    X_sorted = X[idx]
```

```
    y_sorted = y[idx]
```

```
    shards = []
```

```
    chunk_size = math.ceil(len(X) / world_size)
```

```

for rank in range(world_size):
    start = rank * chunk_size
    end = min(start + chunk_size, len(X))

    shards.append({
        "rank": rank,
        "X": X_sorted[start:end],
        "y": y_sorted[start:end],
    })

return shards

```

Then replace:

```
shards = shard_data(X_train, y_train, world_size)
```

with:

```
shards = shard_data_non_iid(X_train, y_train, world_size)
```

Discussion:

How does non-IID data affect local gradients?

Does weight averaging become less stable?

Why is non-IID data a challenge in federated learning?

Extension C: Synchronization Frequency

Instead of averaging every epoch, students can synchronize every k local steps.

```

def train_weight_averaging_sync_frequency(
    X_train,
    y_train,

```

```

X_test,
y_test,
world_size=4,
lr=LEARNING_RATE,
rounds=20,
local_steps=1,
):
    """
    Study how synchronization frequency affects convergence.
    """

```

```

return train_weight_averaging(
    X_train,
    y_train,
    X_test,
    y_test,
    world_size=world_size,
    lr=lr,
    rounds=rounds,
    local_steps=local_steps,
)

```

Test:

```

for local_steps in [1, 2, 5, 10, 20]:
    params, history, elapsed = train_weight_averaging(
        X_train,
        y_train,

```

```

X_test,
y_test,
world_size=4,
rounds=20,
local_steps=local_steps,
)

print(
    local_steps,
    history[-1]["train_loss"],
    history[-1]["test_acc"],
    history[-1]["communication_rounds"],
)

```

Discussion:

Fewer synchronizations reduce communication.

But do they hurt model quality?

Where is the tradeoff?

Extension D: PyTorch DDP Conceptual Skeleton

This helps students see what real DDP looks like.

```
"""
```

Conceptual PyTorch DDP skeleton.

This is for explanation only. It may require a proper torch distributed environment to run.

```
"""
```

```
import os

import torch

import torch.distributed as dist

import torch.nn as nn

import torch.optim as optim

from torch.nn.parallel import DistributedDataParallel as DDP


def ddp_worker(rank, world_size):

    os.environ["MASTER_ADDR"] = "localhost"

    os.environ["MASTER_PORT"] = "29500"


    dist.init_process_group(

        backend="gloo",

        rank=rank,

        world_size=world_size,

    )


    model = nn.Linear(20, 2)

    ddp_model = DDP(model)


    optimizer = optim.SGD(

        ddp_model.parameters(),

        lr=0.01,

    )
```

```
loss_fn = nn.CrossEntropyLoss()
```

```
# Each rank should use a different data shard.
```

```
x = torch.randn(32, 20)
```

```
y = torch.randint(0, 2, (32,))
```

```
optimizer.zero_grad()
```

```
logits = ddp_model(x)
```

```
loss = loss_fn(logits, y)
```

```
loss.backward()
```

```
# DDP synchronizes gradients during backward.
```

```
optimizer.step()
```

```
dist.destroy_process_group()
```

PyTorch's DDP documentation describes using multiple processes with model replicas and synchronizing gradients during the backward pass. The DDP design note also explains that DDP broadcasts model state from rank 0 during construction and uses a reducer to synchronize gradients. [[multiproce...thedocs.io](#)] [[uwpce-pyth....github.io](#)]

Deliverables

Each student submits the following to my email, lidongvtemail@gmail.com

1. Results Table

Must include:

- method
- runtime

- final training loss
- train accuracy
- test accuracy
- communication rounds
- communication scalars

2. Loss Curve

Required:

serial vs gradient averaging across world sizes

3. Short Reflection

Suggested prompt:

In 1–2 paragraphs:

1. How does data-parallel training work?
2. Why do we average gradients?
3. Did gradient averaging match serial training?
4. How did world size affect communication cost?
5. How is weight averaging different from gradient averaging?
6. How could this idea support your final project?